

Sprint 23 Structured Notes: Core Data Structures

0. What Sprint 23 Is About

Sprint 23 is about learning how data can be stored and organized in different ways.

Before this sprint, you learned many JavaScript tools:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- array methods
- Sets and Maps
- DOM basics
- events
- accessibility
- forms
- dates and media APIs
- CRUD
- browser storage
- debugging
- regex
- classes
- Big O basics
- searching
- recursion
- divide and conquer
- merge sort

Sprint 23 adds a new question:

How should the data be stored so the program can use it efficiently?

Earlier sprints taught you how to process data. Sprint 23 teaches you how to choose a structure for storing data.

The main goal of Sprint 23 is:

Understand common storage structures and their tradeoffs.

By the end of Sprint 23, you should understand:

- data structures
- why data structure choice matters
- static arrays
- dynamic arrays
- common array operations
- time complexity of common array operations
- stacks
- LIFO order
- stack operations
- stack implementation with JavaScript arrays
- queues
- FIFO order
- queue operations
- simple queue implementation with arrays
- better queue implementation with a head pointer
- linked lists
- nodes
- head
- tail
- next references
- singly linked lists
- doubly linked lists
- previous references
- how linked lists differ from arrays
- how to choose the right data structure
- beginner mistakes with data structures
- practice problems for Sprint 23

Part A: Current Progress Before Sprint 23

1. What You Already Know

Before Sprint 23, you already know enough JavaScript to understand data structures.

Earlier area	What you learned	How it helps in Sprint 23
Variables	Store values with <code>let</code> and <code>const</code>	Data structures are stored in variables
Types	Understand strings, numbers, booleans, objects, arrays, <code>null</code> , and <code>undefined</code>	Data structures contain values of different types

Earlier area	What you learned	How it helps in Sprint 23
Functions	Write reusable logic	Data structures usually have methods like <code>push</code> , <code>pop</code> , <code>enqueue</code> , and <code>dequeue</code>
Arrays	Store ordered lists	Arrays are the first data structure you learned
Objects	Store key-value data	Nodes are usually represented as objects or classes
Classes	Create reusable object blueprints	Stacks, queues, and linked lists can be written as classes
Loops	Repeat work	Linked lists need loops for traversal
Big O	Think about efficiency	Data structures are compared by operation cost
Searching and sorting	Find and organize data	Data structures affect how searching and organizing work

Simple summary:

You already know how to write JavaScript. Sprint 23 teaches you how to organize data so your JavaScript can work with it better.

Part B: The Core Idea of Data Structures

2. What Is a Data Structure?

A data structure is a way to organize data.

Simple meaning:

A data structure is a container with rules for how data is stored, added, removed, and accessed.

Examples:

```
const names = ["Ada", "Grace", "Linus"];
```

This is an array. It stores values in order.

Another example:

```
const user = {  
  name: "Ada",  
  score: 95  
};
```

This is an object. It stores values using property names.

Sprint 23 focuses on these structures:

- arrays
- stacks
- queues
- singly linked lists
- doubly linked lists

3. Why Data Structures Matter

Different data structures are good at different jobs.

A structure that is good for one task may be weak for another task.

Example:

An array is good for direct index access:

```
const scores = [80, 90, 100];  
  
console.log(scores[1]); // 90
```

Accessing `scores[1]` is fast because arrays are index-based.

But removing from the front of a JavaScript array can be less efficient:

```
const scores = [80, 90, 100];  
  
scores.shift();  
  
console.log(scores); // [90, 100]
```

After removing the first item, JavaScript may need to move the remaining items.

The main question is not:

Which data structure is always best?

The better question is:

What operations will I do most often?

Common operations:

- access an item
 - add an item
 - remove an item
 - search for an item
 - process the newest item first
 - process the oldest item first
 - move forward through data
 - move backward through data
-

4. Operation Tradeoffs

A tradeoff means one choice gives you one advantage but may create another weakness.

Example:

Arrays give fast direct access by index:

```
arr[0]  
arr[5]  
arr[100]
```

But arrays are not always best for frequent insertion or removal in the middle.

Linked lists can be useful for reference-based insertion and removal, but they do not give simple direct index access.

Simple rule:

A data structure is chosen based on the operations you need most.

Part C: Arrays as Data Structures

5. What Is an Array?

An array is an ordered collection of values.

Example:

```
const fruits = ["apple", "banana", "cherry"];
```

The array stores values in order:

Index	Value
0	"apple"
1	"banana"
2	"cherry"

Arrays are zero-indexed.

Simple meaning:

The first item is at index `0`, not index `1`.

6. Static Arrays

A static array has a fixed size.

Simple meaning:

A static array has a set number of slots.

Mental model:

```
[10, 20, 30]
```

This array has exactly 3 slots.

In lower-level programming languages, a static array often has a fixed amount of memory. That means you decide the size first.

Conceptual JavaScript example:

```
const scores = new Array(3);

scores[0] = 90;
scores[1] = 85;
scores[2] = 100;

console.log(scores[1]); // 85
```

This creates an array with 3 empty positions first.

Important JavaScript note:

JavaScript arrays are dynamic by default. `new Array(3)` can help you understand the idea of a fixed-size array, but JavaScript arrays can still grow.

Example:

```
const scores = new Array(3);

scores[0] = 90;
scores[1] = 85;
scores[2] = 100;
scores.push(70);

console.log(scores); // [90, 85, 100, 70]
```

Even though the array started with 3 slots, JavaScript allowed another item to be added.

7. Dynamic Arrays

A dynamic array can grow or shrink.

Simple meaning:

A dynamic array changes size as your program adds or removes items.

JavaScript arrays are dynamic.

Example:

```
const tasks = [];

tasks.push("Study");
tasks.push("Practice");
tasks.push("Review");

console.log(tasks);
// ["Study", "Practice", "Review"]
```

You did not need to decide the final size first.

You can also remove items:

```
const tasks = ["Study", "Practice", "Review"];

const lastTask = tasks.pop();

console.log(lastTask); // Review
console.log(tasks);    // ["Study", "Practice"]
```

8. Common Array Operations

Operation	JavaScript example	Simple meaning
Access by index	<code>arr[2]</code>	Get item at position 2
Update by index	<code>arr[2] = value</code>	Change item at position 2
Add to end	<code>arr.push(value)</code>	Put value at the back
Remove from end	<code>arr.pop()</code>	Remove the last value
Add to front	<code>arr.unshift(value)</code>	Put value at the front
Remove from front	<code>arr.shift()</code>	Remove the first value
Insert in middle	<code>arr.splice(index, 0, value)</code>	Add value at a chosen position
Remove from middle	<code>arr.splice(index, 1)</code>	Remove value from a chosen position

9. Time Complexity of Common Array Operations

Operation	Typical cost	Why
Access by index	$O(1)$	JavaScript can go directly to the index
Update by index	$O(1)$	JavaScript can update a known position
Add to end with <code>push()</code>	Usually $O(1)$	Add after the last item
Remove from end with <code>pop()</code>	$O(1)$	Remove the last item directly
Add to front with <code>unshift()</code>	$O(n)$	Existing items may need to shift right
Remove from front with <code>shift()</code>	$O(n)$	Existing items may need to shift left
Insert in middle with <code>splice()</code>	$O(n)$	Items after the index may need to move
Remove from middle with <code>splice()</code>	$O(n)$	Items after the index may need to move

Simple rule:

Arrays are strong for index access and end operations. They are weaker when many items must move.

10. Why Middle Insertion Can Be $O(n)$

Example:

```
const numbers = [10, 20, 40];  
  
numbers.splice(2, 0, 30);  
  
console.log(numbers);  
// [10, 20, 30, 40]
```

What happened?

Before:

10, 20, 40

Insert `30` before `40`:

```
10, 20, 30, 40
```

JavaScript had to make room before `40`.

If the array is very large, many values may need to move.

That is why this can be `O(n)`.

11. When Arrays Are a Good Choice

Use an array when:

- you need an ordered list
- you need direct access by index
- you mostly add/remove at the end
- you need built-in methods like `map`, `filter`, `reduce`, `sort`, and `forEach`
- you are building normal JavaScript app data

Examples:

```
const names = ["Ada", "Grace", "Linus"];  
const scores = [90, 85, 100];  
const tasks = ["Study", "Code", "Review"];
```

For most everyday JavaScript work, arrays are the first structure to consider.

Part D: Stacks

12. What Is a Stack?

A stack is a data structure where the last item added is the first item removed.

This is called:

Last In, First Out

Short form:

LIFO

Simple meaning:

The newest item comes out first.

Mental model:

Imagine a stack of plates.

You add a plate to the top. You also remove a plate from the top.

The last plate added is the first plate removed.

13. Stack Mental Model

Start with an empty stack:

[]

Push **A** :

[A]

Push **B** :

[A, B]

Push **C** :

[A, B, C]

Now pop:

C comes out first

Then:

B comes out next

Then:

A comes out last

The order is:

Added: A, B, C
Removed: C, B, A

14. Stack With a JavaScript Array

JavaScript arrays can behave like stacks.

Use:

- `push()` to add to the top
- `pop()` to remove from the top

Example:

```
const stack = [];  
  
stack.push("A");  
stack.push("B");  
stack.push("C");  
  
console.log(stack.pop()); // C  
console.log(stack.pop()); // B  
console.log(stack.pop()); // A
```

The last item added was `"C"`, and it came out first.

15. Stack Operations

Operation	Meaning
<code>push(value)</code>	Add a value to the top
<code>pop()</code>	Remove and return the top value
<code>peek()</code>	Look at the top value without removing it
<code>isEmpty()</code>	Check whether the stack has no values

Operation	Meaning
<code>size()</code>	Count how many values are in the stack

16. Stack Class

A stack can be written as a class.

```
class Stack {
  constructor() {
    this.items = [];
  }

  push(value) {
    this.items.push(value);
  }

  pop() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items.pop();
  }

  peek() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items[this.items.length - 1];
  }

  isEmpty() {
    return this.items.length === 0;
  }

  size() {
    return this.items.length;
  }
}
```

Use it:

```
const history = new Stack();

history.push("home");
history.push("profile");
history.push("settings");

console.log(history.peek()); // settings
console.log(history.pop()); // settings
console.log(history.pop()); // profile
console.log(history.size()); // 1
```

17. Stack Dry Run

Code:

```
const history = new Stack();

history.push("home");
history.push("profile");
history.push("settings");

history.pop();
```

Step-by-step:

Step	Stack contents	What happened
1	<code>[]</code>	Stack starts empty
2	<code>["home"]</code>	<code>home</code> is pushed
3	<code>["home", "profile"]</code>	<code>profile</code> is pushed
4	<code>["home", "profile", "settings"]</code>	<code>settings</code> is pushed
5	<code>["home", "profile"]</code>	<code>settings</code> is popped

Result:

```
settings
```

18. Why `peek()` Is Useful

`peek()` lets you look at the top item without removing it.

Example:

```
const stack = new Stack();

stack.push("first");
stack.push("second");

console.log(stack.peek()); // second
console.log(stack.peek()); // second
console.log(stack.size()); // 2
```

The item is still there because `peek()` does not remove it.

19. Stack Empty Behavior

If a stack is empty, `pop()` has no value to remove.

In JavaScript arrays:

```
const items = [];

console.log(items.pop()); // undefined
```

In your own stack class, you can choose how to handle this.

Beginner-friendly choice:

```
if (this.items.length === 0) {
  return null;
}
```

Why `null`?

Because it clearly means:

There is no value here.

20. When to Use a Stack

Use a stack when the newest item should be handled first.

Use case	Why a stack fits
Undo behavior	The most recent action is undone first
Browser back behavior	The last page visited is handled first
Backtracking	The most recent path is checked first
Depth-first search	Go deep first, then come back
Function call stack mental model	The last function called finishes first

Example idea: undo behavior

```
const undoStack = new Stack();

undoStack.push("typed A");
undoStack.push("typed B");
undoStack.push("typed C");

console.log(undoStack.pop()); // typed C
```

The last action is undone first.

Part E: Queues

21. What Is a Queue?

A queue is a data structure where the first item added is the first item removed.

This is called:

First In, First Out

Short form:

FIFO

Simple meaning:

The oldest item comes out first.

Mental model:

Imagine a line at a store.

The first person in line is served first.

22. Queue Mental Model

Start with an empty queue:

[]

Enqueue **A** :

[A]

Enqueue **B** :

[A, B]

Enqueue **C** :

[A, B, C]

Now dequeue:

A comes out first

Then:

B comes out next

Then:

C comes out last

The order is:

Added: A, B, C
Removed: A, B, C

23. Queue With a JavaScript Array

JavaScript arrays can behave like queues.

Use:

- `push()` to add to the back
- `shift()` to remove from the front

Example:

```
const queue = [];  
  
queue.push("A");  
queue.push("B");  
queue.push("C");  
  
console.log(queue.shift()); // A  
console.log(queue.shift()); // B  
console.log(queue.shift()); // C
```

The first item added was `"A"`, and it came out first.

24. Queue Operations

Operation	Meaning
<code>enqueue(value)</code>	Add a value to the back
<code>dequeue()</code>	Remove and return the front value
<code>peek()</code>	Look at the front value without removing it
<code>isEmpty()</code>	Check whether the queue has no values
<code>size()</code>	Count how many values are in the queue

25. Simple Queue Class

This version is easy to understand.

```
class SimpleQueue {
  constructor() {
    this.items = [];
  }

  enqueue(value) {
    this.items.push(value);
  }

  dequeue() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items.shift();
  }

  peek() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items[0];
  }

  isEmpty() {
    return this.items.length === 0;
  }

  size() {
    return this.items.length;
  }
}
```

Use it:

```
const line = new SimpleQueue();

line.enqueue("Ada");
line.enqueue("Grace");
line.enqueue("Linus");
```

```
console.log(line.dequeue()); // Ada
console.log(line.dequeue()); // Grace
console.log(line.peek());    // Linus
```

26. Problem With `shift()` in Large Queues

The simple queue uses `shift()`.

```
return this.items.shift();
```

This is clear, but it can be inefficient for large arrays.

Why?

When the first item is removed, the remaining items may need to move forward.

Example:

Before shift:

[A, B, C, D]

After shift:

[B, C, D]

The indexes changed:

B was index 1, now B is index 0

C was index 2, now C is index 1

D was index 3, now D is index 2

For small beginner projects, this is usually fine.

For larger queues, use a different approach.

27. Better Queue With a Head Pointer

A better queue can avoid using `shift()`.

Instead of physically removing the first item, it keeps a `head` index.

Simple meaning:

`head` tells us where the current front of the queue is.

```
class Queue {
  constructor() {
    this.items = [];
    this.head = 0;
  }

  enqueue(value) {
    this.items.push(value);
  }

  dequeue() {
    if (this.isEmpty()) {
      return null;
    }

    const value = this.items[this.head];
    this.head++;

    return value;
  }

  peek() {
    if (this.isEmpty()) {
      return null;
    }

    return this.items[this.head];
  }

  isEmpty() {
    return this.head >= this.items.length;
  }

  size() {
    return this.items.length - this.head;
  }
}
```

Use it:

```
const supportQueue = new Queue();

supportQueue.enqueue("Ticket 1");
supportQueue.enqueue("Ticket 2");
supportQueue.enqueue("Ticket 3");

console.log(supportQueue.dequeue()); // Ticket 1
console.log(supportQueue.peek());    // Ticket 2
console.log(supportQueue.size());     // 2
```

28. Queue With Head Pointer Dry Run

Code:

```
const queue = new Queue();

queue.enqueue("A");
queue.enqueue("B");
queue.enqueue("C");

queue.dequeue();
```

Initial state:

```
items = []
head = 0
```

After enqueue operations:

```
items = ["A", "B", "C"]
head = 0
```

`dequeue()` reads:

```
this.items[this.head]
```

Since `head` is `0`, it reads:

```
this.items[0]
```

That is:

A

Then `head` becomes `1`.

New state:

```
items = ["A", "B", "C"]  
head = 1
```

The array still contains `"A"`, but the queue now treats `"B"` as the front.

29. When to Use a Queue

Use a queue when the oldest item should be handled first.

Use case	Why a queue fits
Customer service line	First customer is served first
Print jobs	First document prints first
Support tickets	Oldest ticket should be handled first
Task scheduling	Earlier task is processed first
Breadth-first search	Visit nearest items first

Example idea: support tickets

```
const tickets = new Queue();  
  
tickets.enqueue("Ticket A");  
tickets.enqueue("Ticket B");  
tickets.enqueue("Ticket C");  
  
console.log(tickets.dequeue()); // Ticket A
```

The first ticket is handled first.

Part F: Stack vs Queue

30. Stack and Queue Comparison

Structure	Order rule	Add operation	Remove operation	Real-life model
Stack	LIFO	Add to top	Remove from top	Stack of plates
Queue	FIFO	Add to back	Remove from front	Line at a store

Stack:

Added: A, B, C
Removed: C, B, A

Queue:

Added: A, B, C
Removed: A, B, C

Simple rule:

Use a stack for newest-first behavior. Use a queue for oldest-first behavior.

Part G: Linked Lists

31. What Is a Linked List?

A linked list is a data structure made of nodes.

Each node stores:

- a value
- a reference to another node

Simple meaning:

A linked list is a chain of nodes.

Mental model:


```
10 -> 20 -> 30 -> null
```

Each box is a node.

Each arrow points to the next node.

`null` means the list has ended.

32. Array vs Linked List

An array stores values by index:

```
const numbers = [10, 20, 30];  
  
console.log(numbers[1]); // 20
```

A linked list stores values in nodes:

```
10 -> 20 -> 30 -> null
```

To reach `30`, you usually start at the first node and follow the arrows.

Comparison:

Feature	Array	Linked list
Direct index access	Yes	No
Uses indexes	Yes	No, not naturally
Made of nodes	No	Yes
Traversal	By index or loop	By following references
First item	<code>arr[0]</code>	<code>head</code>
Last item	<code>arr[arr.length - 1]</code>	<code>tail</code> , if tracked

33. What Is a Node?

A node is one piece of a linked list.

A singly linked list node usually has:

- value
- next

Example object:

```
const node = {  
  value: 10,  
  next: null  
};
```

Simple meaning:

value stores the data. next points to the next node.

34. Node Class

A node can be written as a class.

```
class Node {  
  constructor(value) {  
    this.value = value;  
    this.next = null;  
  }  
}
```

Use it:

```
const node = new Node("A");  
  
console.log(node.value); // A  
console.log(node.next); // null
```

At first, next is null because the node is not connected to another node yet.

35. Connecting Nodes

You can connect one node to another using next .

```
const first = new Node("A");
const second = new Node("B");
const third = new Node("C");

first.next = second;
second.next = third;

console.log(first.next.value);      // B
console.log(first.next.next.value); // C
```

Mental model:

```
A -> B -> C -> null
```

`first.next` points to `second`.

`second.next` points to `third`.

`third.next` is still `null`.

Part H: Singly Linked Lists

36. What Is a Singly Linked List?

A singly linked list is a linked list where each node points only to the next node.

Simple meaning:

You can move forward through the list, one node at a time.

Mental model:

```
head
↓
A -> B -> C -> null
```

The arrows go one direction.

37. Important Singly Linked List Words

Word	Meaning
head	The first node in the list
tail	The last node in the list
value	The data inside a node
next	A reference to the next node
null	Means there is no next node
traversal	Moving through the list node by node

38. Empty Singly Linked List

An empty linked list has no nodes.

```
head = null
tail = null
```

Simple meaning:

There is no first node and no last node.

Class setup:

```
class SinglyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
  }
}
```

39. Singly Linked List Class With `append()` and `find()`

```
class Node {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}
```

```

}

class SinglyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
  }

  append(value) {
    const newNode = new Node(value);

    if (this.head === null) {
      this.head = newNode;
      this.tail = newNode;
      return;
    }

    this.tail.next = newNode;
    this.tail = newNode;
  }

  find(value) {
    let current = this.head;

    while (current !== null) {
      if (current.value === value) {
        return current;
      }

      current = current.next;
    }

    return null;
  }

  print() {
    let current = this.head;
    const values = [];

    while (current !== null) {
      values.push(current.value);
      current = current.next;
    }

    console.log(values.join(" -> "));
  }
}

```

Use it:

```
const list = new SinglyLinkedList();

list.append("A");
list.append("B");
list.append("C");

list.print(); // A -> B -> C

console.log(list.find("B"));
// Node { value: "B", next: Node {...} }

console.log(list.find("X"));
// null
```

40. How `append()` Works

`append()` adds a new node to the end of the list.

Code:

```
append(value) {
  const newNode = new Node(value);

  if (this.head === null) {
    this.head = newNode;
    this.tail = newNode;
    return;
  }

  this.tail.next = newNode;
  this.tail = newNode;
}
```

There are two cases.

Case 1: The list is empty

```
if (this.head === null) {
  this.head = newNode;
  this.tail = newNode;
}
```

```
    return;  
}
```

If the list is empty, the new node becomes both:

- the head
- the tail

Mental model:

```
head  
↓  
A  
↑  
tail
```

Case 2: The list already has nodes

```
this.tail.next = newNode;  
this.tail = newNode;
```

This connects the old tail to the new node.

Then the new node becomes the tail.

Mental model before adding **C**:

```
head      tail  
↓         ↓  
A  ->    B
```

After adding **C**:

```
head      tail  
↓         ↓  
A  ->  B  ->  C
```

41. `append()` Dry Run

Start:

```
head = null  
tail = null
```

Run:

```
list.append("A");
```

Result:

```
head  
↓  
A  
↑  
tail
```

Run:

```
list.append("B");
```

Result:

```
head      tail  
↓         ↓  
A  ->    B
```

Run:

```
list.append("C");
```

Result:

```
head      tail  
↓         ↓  
A  ->  B  ->  C
```


42. How `find()` Works

`find()` searches the list from the head to the end.

Code:

```
find(value) {  
  let current = this.head;  
  
  while (current !== null) {  
    if (current.value === value) {  
      return current;  
    }  
  
    current = current.next;  
  }  
  
  return null;  
}
```

Step-by-step:

1. Start at the head.
2. Check the current node's value.
3. If it matches, return the node.
4. If it does not match, move to the next node.
5. Stop when `current` becomes `null`.
6. Return `null` if the value was not found.

43. `find()` Dry Run

List:

```
A -> B -> C -> null
```

Run:

```
list.find("C");
```

Dry run:

Step	current.value	Match?	What happens
1	"A"	No	Move to next node
2	"B"	No	Move to next node
3	"C"	Yes	Return current node

Result:

Node with value "C"

Run:

```
list.find("X");
```

Dry run:

Step	current.value	Match?	What happens
1	"A"	No	Move to next node
2	"B"	No	Move to next node
3	"C"	No	Move to next node
4	null	Not possible	Stop loop

Result:

null

44. How print() Works

print() walks through the linked list and collects values.

Code:

```
print() {
  let current = this.head;
  const values = [];

  while (current !== null) {
```

```
    values.push(current.value);
    current = current.next;
  }

  console.log(values.join(" -> "));
}
```

Simple meaning:

Start at the first node, collect each value, and move forward until the list ends.

Example:

A -> B -> C

Output:

A -> B -> C

45. Singly Linked List Time Complexity

Operation	Typical cost	Why
Add to end with tail	$O(1)$	Tail points directly to the last node
Find a value	$O(n)$	You may need to check every node
Print all values	$O(n)$	You must visit every node
Access by index	$O(n)$	You must walk from the head to that position

Important point:

Linked lists do not have direct index access like arrays.

This is fast in an array:

arr[5]

But in a linked list, you usually must start at `head` and move node by node.

Part I: Doubly Linked Lists

46. What Is a Doubly Linked List?

A doubly linked list is a linked list where each node points in two directions.

Each node can point to:

- the next node
- the previous node

Simple meaning:

A doubly linked list lets you move forward and backward.

Mental model:

```
null <- A <-> B <-> C -> null
```

47. Doubly Linked List Node

A doubly linked list node usually has:

- value
- next
- previous

Example:

```
class DoublyNode {  
  constructor(value) {  
    this.value = value;  
    this.next = null;  
    this.previous = null;  
  }  
}
```

Simple meaning:

- value stores the data
- next points forward
- previous points backward

48. Singly vs Doubly Linked List

Feature	Singly linked list	Doubly linked list
Direction	Forward only	Forward and backward
Node references	<code>next</code>	<code>next</code> and <code>previous</code>
Memory per node	Less	More
Backward movement	Not direct	Direct
Simpler to implement	Yes	More complex

Simple rule:

A doubly linked list gives more navigation power but uses more memory and has more references to manage.

49. Connecting Doubly Linked Nodes

Example:

```
const first = new DoublyNode("A");
const second = new DoublyNode("B");
const third = new DoublyNode("C");

first.next = second;
second.previous = first;

second.next = third;
third.previous = second;
```

Mental model:

A <-> B <-> C

Check connections:

```
console.log(first.next.value); // B
console.log(second.previous.value); // A
console.log(second.next.value); // C
console.log(third.previous.value); // B
```

50. When to Use a Doubly Linked List

Use a doubly linked list when you need to move both forward and backward.

Use case	Why it fits
Browser history	Move back and forward
Music playlist	Move to previous or next song
Image carousel	Move to previous or next image
Text editor states	Move between previous and next states
Navigation systems	Move both directions through items

Example mental model for a music playlist:

```
Previous song <- Current song -> Next song
```

A doubly linked list fits this better than a singly linked list.

Part J: Choosing the Right Data Structure

51. Main Comparison Table

Data structure	Main rule	Good for	Weakness
Dynamic array	Ordered list with indexes	General JavaScript lists	Front/middle insertions and removals can be costly
Stack	LIFO	Undo, backtracking, newest-first work	Only top item is directly used
Queue	FIFO	Lines, scheduling, oldest-first work	Simple JS queue with <code>shift()</code> can be inefficient
Singly linked list	One-way node chain	Forward traversal, node-reference practice	No direct index access
Doubly linked list	Two-way node chain	Forward/backward navigation	Uses more memory and complexity

52. Choosing by Operation

If you need to...	Consider using...	Reason
Access items by index	Array	Index access is simple and fast
Add/remove from the end	Array or stack	<code>push()</code> and <code>pop()</code> are natural
Undo the latest action	Stack	Last action should be first undone
Process oldest request first	Queue	First request should be handled first
Move through nodes one by one	Singly linked list	Nodes point forward
Move forward and backward	Doubly linked list	Nodes point both ways
Build normal app lists	Array	JavaScript arrays are practical and built-in

53. Practical JavaScript Rule

For most beginner and everyday JavaScript projects:

Start with arrays unless you have a clear reason not to.

Use stacks when you specifically need LIFO behavior.

Use queues when you specifically need FIFO behavior.

Use linked lists mainly to learn data-structure thinking, reference behavior, and operation tradeoffs.

Part K: Common Beginner Mistakes

54. Mistake 1: Thinking One Data Structure Is Always Best

Wrong idea:

Arrays are always best.

Better idea:

Arrays are often best in everyday JavaScript, but the right structure depends on the needed operations.

55. Mistake 2: Confusing Stack and Queue

Stack:

```
Last in, first out
```

Queue:

```
First in, first out
```

Memory trick:

- Stack = stack of plates
 - Queue = line of people
-

56. Mistake 3: Using `shift()` Without Understanding Cost

This is easy:

```
queue.shift();
```

But for large queues, it can be inefficient.

Better large-queue idea:

```
this.head++;
```

The head-pointer approach avoids repeatedly shifting all items.

57. Mistake 4: Expecting Linked Lists to Have Fast Index Access

Arrays allow this:

```
arr[10]
```

Linked lists do not naturally work this way.

In a linked list, you usually start at `head` and follow `next` references.

58. Mistake 5: Forgetting to Update `tail`

When appending to a linked list, this is important:

```
this.tail.next = newNode;  
this.tail = newNode;
```

If you forget to update `tail`, the list may not correctly know its last node.

59. Mistake 6: Forgetting the Empty List Case

This is necessary:

```
if (this.head === null) {  
  this.head = newNode;  
  this.tail = newNode;  
  return;  
}
```

Without this, appending to an empty linked list can break.

60. Mistake 7: Infinite Loops in Linked List Traversal

This loop is normal:

```
while (current !== null) {  
  current = current.next;  
}
```

But if you forget this line:

```
current = current.next;
```

The loop may never end.

Part L: Sprint 23 Code Review Checklist

Before you say your Sprint 23 code works, check these questions.

Array checklist

- Can I explain whether the array is static or dynamic?
- Am I using indexes correctly?
- Do I understand that `push()` and `pop()` are usually efficient?
- Do I understand that `shift()`, `unshift()`, and middle `splice()` may require moving items?

Stack checklist

- Does my stack use LIFO order?
- Does `push()` add to the top?
- Does `pop()` remove from the top?
- Does `peek()` return the top without removing it?
- Does empty `pop()` behave safely?

Queue checklist

- Does my queue use FIFO order?
- Does `enqueue()` add to the back?
- Does `dequeue()` remove from the front?
- Does `peek()` return the front without removing it?
- Am I using `shift()` only when the queue is small or simplicity matters?
- For a better queue, am I using a head pointer correctly?

Linked list checklist

- Does each node have a value?
- Does each singly linked node have `next`?
- Does the list have a `head`?
- Does the list update `tail` correctly?
- Does traversal move `current = current.next`?
- Does the code handle an empty list?
- Does `find()` return `null` when the value is missing?

Doubly linked list checklist

- Does each node have `next` and `previous`?
- Are forward links correct?
- Are backward links correct?
- Do I understand that doubly linked lists use more memory per node?

Part M: Practice Tasks

Practice 1: Use an Array as a Stack

Create a stack with an array.

Requirements:

- push three values
- pop one value
- print the remaining stack

Starter:

```
const stack = [];  
  
stack.push("A");  
stack.push("B");  
stack.push("C");  
  
console.log(stack.pop());  
console.log(stack);
```

Expected idea:

```
C  
["A", "B"]
```

Practice 2: Use an Array as a Queue

Create a queue with an array.

Requirements:

- enqueue three values with `push()`
- dequeue one value with `shift()`
- print the remaining queue

Starter:

```
const queue = [];  
  
queue.push("A");
```

```
queue.push("B");
queue.push("C");

console.log(queue.shift());
console.log(queue);
```

Expected idea:

```
A
["B", "C"]
```

Practice 3: Create a Stack Class

Build this class:

```
class Stack {
  constructor() {
    this.items = [];
  }

  push(value) {
    this.items.push(value);
  }

  pop() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items.pop();
  }

  peek() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items[this.items.length - 1];
  }
}
```

Test:

```
const stack = new Stack();

stack.push(10);
stack.push(20);

console.log(stack.peek()); // 20
console.log(stack.pop()); // 20
console.log(stack.pop()); // 10
console.log(stack.pop()); // null
```

Practice 4: Create a Queue Class

Build this class:

```
class Queue {
  constructor() {
    this.items = [];
    this.head = 0;
  }

  enqueue(value) {
    this.items.push(value);
  }

  dequeue() {
    if (this.isEmpty()) {
      return null;
    }

    const value = this.items[this.head];
    this.head++;
    return value;
  }

  peek() {
    if (this.isEmpty()) {
      return null;
    }

    return this.items[this.head];
  }

  isEmpty() {
    return this.head >= this.items.length;
  }
}
```

```
}  
}
```

Test:

```
const queue = new Queue();  
  
queue.enqueue("A");  
queue.enqueue("B");  
queue.enqueue("C");  
  
console.log(queue.dequeue()); // A  
console.log(queue.peek());    // B  
console.log(queue.dequeue()); // B  
console.log(queue.dequeue()); // C  
console.log(queue.dequeue()); // null
```

Practice 5: Create a Node

Create one node.

```
class Node {  
  constructor(value) {  
    this.value = value;  
    this.next = null;  
  }  
}  
  
const node = new Node("A");  
  
console.log(node.value); // A  
console.log(node.next);  // null
```

Then connect two nodes:

```
const first = new Node("A");  
const second = new Node("B");  
  
first.next = second;  
  
console.log(first.next.value); // B
```

Practice 6: Create a Singly Linked List

Build a linked list with `append`, `find`, and `print`.

```
class Node {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}

class SinglyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
  }

  append(value) {
    const newNode = new Node(value);

    if (this.head === null) {
      this.head = newNode;
      this.tail = newNode;
      return;
    }

    this.tail.next = newNode;
    this.tail = newNode;
  }

  find(value) {
    let current = this.head;

    while (current !== null) {
      if (current.value === value) {
        return current;
      }

      current = current.next;
    }

    return null;
  }

  print() {
    let current = this.head;
```

```

const values = [];

while (current !== null) {
  values.push(current.value);
  current = current.next;
}

console.log(values.join(" -> "));
}
}

```

Test:

```

const list = new SinglyLinkedList();

list.append("A");
list.append("B");
list.append("C");

list.print(); // A -> B -> C

console.log(list.find("B")); // Node with value B
console.log(list.find("X")); // null

```

Practice 7: Create Doubly Linked Nodes

Create and connect doubly linked nodes.

```

class DoublyNode {
  constructor(value) {
    this.value = value;
    this.next = null;
    this.previous = null;
  }
}

const first = new DoublyNode("A");
const second = new DoublyNode("B");
const third = new DoublyNode("C");

first.next = second;
second.previous = first;

```



```
second.next = third;
third.previous = second;

console.log(first.next.value);    // B
console.log(second.previous.value); // A
console.log(second.next.value);   // C
console.log(third.previous.value); // B
```

Part N: Mini Project Ideas

Mini Project 1: Undo Stack

Build a small undo system.

Requirements:

- create a stack
- push user actions into the stack
- pop the last action when undo is called

Example:

```
const undoStack = new Stack();

undoStack.push("typed hello");
undoStack.push("made text bold");
undoStack.push("changed color");

console.log(`Undo: ${undoStack.pop()}`);
console.log(`Undo: ${undoStack.pop()}`);
```

Expected idea:

```
Undo: changed color
Undo: made text bold
```

Mini Project 2: Support Ticket Queue

Build a support ticket system.

Requirements:

- create a queue
- enqueue tickets
- dequeue tickets in the order they arrived

Example:

```
const tickets = new Queue();

tickets.enqueue("Ticket 1: Login problem");
tickets.enqueue("Ticket 2: Payment problem");
tickets.enqueue("Ticket 3: Profile problem");

console.log(tickets.dequeue());
console.log(tickets.dequeue());
```

Expected idea:

```
Ticket 1: Login problem
Ticket 2: Payment problem
```

Mini Project 3: Simple Playlist With Doubly Linked Nodes

Create three songs and connect them forward and backward.

Example:

```
const song1 = new DoublyNode("Song A");
const song2 = new DoublyNode("Song B");
const song3 = new DoublyNode("Song C");

song1.next = song2;
song2.previous = song1;

song2.next = song3;
song3.previous = song2;

console.log(song2.previous.value); // Song A
console.log(song2.value);          // Song B
console.log(song2.next.value);     // Song C
```

Part O: Sprint 23 Minimum Standard

You understand Sprint 23 when you can explain these without looking:

1. A data structure is a way to organize data.
 2. A static array has a fixed-size idea.
 3. JavaScript arrays are dynamic by default.
 4. Arrays are strong for direct index access.
 5. `push()` adds to the end of an array.
 6. `pop()` removes from the end of an array.
 7. `shift()` removes from the front, but can be inefficient for large arrays.
 8. A stack uses LIFO order.
 9. LIFO means last in, first out.
 10. A queue uses FIFO order.
 11. FIFO means first in, first out.
 12. `push()` and `pop()` naturally create stack behavior.
 13. `push()` and `shift()` can create simple queue behavior.
 14. A better queue can use a head pointer.
 15. A linked list is made of nodes.
 16. A node stores a value and reference information.
 17. A singly linked list node has `value` and `next`.
 18. A doubly linked list node has `value`, `next`, and `previous`.
 19. `head` means the first node.
 20. `tail` means the last node.
 21. `null` marks the end of a linked list.
 22. Linked lists do not have direct index access like arrays.
 23. Singly linked lists move forward.
 24. Doubly linked lists move forward and backward.
 25. Data structure choice depends on the operations you need most.
-

Part P: Final Sprint 23 Summary

Sprint 23 teaches that storing data is a design decision.

Arrays are practical and common in JavaScript. They are strong when you need ordered data and direct index access.

Stacks are useful when the newest item should be handled first.

Queues are useful when the oldest item should be handled first.

Linked lists are made of nodes connected by references. A singly linked list moves forward. A doubly linked list moves forward and backward.

The most important Sprint 23 idea is:

Do not choose a data structure by habit. Choose it based on the operations your program needs most.